An Energy-Aware Prompt Engineering Framework for Sustainable LLM Inference

1st Abdullah Al Masud Bhuiyan
Computer Science and Engineering
United International University
Dhaka, Bangladesh
abhuiyan221074@bscse.uiu.ac.bd

2nd Kaium Al Limon
Computer Science and Engineering
United International University
Dhaka, Bangladesh
klimon221291@bscse.uiu.ac.bd

3rd Sharmin Sultana Liza
Computer Science and Engineering
United International University
Dhaka, Bangladesh
sliza221331@bscse.uiu.ac.bd

4th Md. Rahamat Ullah
Computer Science and Engineering
United International University
Dhaka, Bangladesh
mullah221325@bscse.uiu.ac.bd

5th Apurba Kumar Paul
Computer Science and Engineering
United International University
Dhaka, Bangladesh
apaul221320@bscse.uiu.ac.bd

***Abstract*—Large Language Models are essential building blocks of many artificial intelligence solutions, including question answering, summarization, text generation, and programming. However, despite the high efficiency, large language models are computationally expensive, making inference associated with elevated levels of energy usage, expenses, and negative impacts on the environment. In this paper, we introduce an Energy-Aware Prompt Engineering Framework for optimizing Large Language Model inference based on complexity-aware prompt optimization and adaptive prompt generation. We describe the proposed framework for analyzing the complexity of queries, semantic decomposition, prompt compression, quality estimation of the optimized prompts via embeddings' proximity, and adapting the inference parameters based on the specific nature of the query. An Energy-Aware Prompt Engineering Framework is implemented with a multi-layered structure, including analysis of the complexity of requests, prompt decomposition, prompt optimization, validation of the optimization results, adaptive prompt generation, execution of the optimized inference process, performance monitoring, and evaluation. Our experiments on several classes of benchmarks show an average decrease of 69.8% in the number of tokens used and 65.2% in the latency of inference while producing responses of satisfactory quality in most cases.**

***Index Terms*—Prompt Engineering, Green AI, Sustainable Artificial Intelligence, Energy-Efficient Computing, Prompt Optimization, Adaptive Inference, Query Complexity Analysis, Token Reduction, Energy-Aware Systems.**

I. Introduction

These models have changed the entire landscape of artificial intelligence through their superior abilities in natural language processing and generation. Some of the large language models include GPT, LLaMA, Qwen, and more, which are being increasingly used in areas like conversational agents, learning environments, content generators, and code editors, among others. Though these models generate superior output, the computational resources that they consume for inference are still considerable.

The usage of LLM services by many people is raising concerns about energy efficiency and sustainability. Prompts that are too long, unnecessary use of reasoning chains, repeated contexts, and an unnecessarily high token budget result in higher computational load. Inference cost depends linearly on the number of tokens processed. This means that an ineffectively created prompt can cause a waste of computational resources.

Green AI researchers have focused their efforts on making the models more efficient without compromising their performance. Model compression and quantization have been the primary focus of Green AI, while there is an unexplored path in the form of prompt engineering that can be utilized to make the models less costly to run. Prompt optimization is possible as it doesn't change the model's internal structure.

This paper presents a novel framework for Energy-Aware Prompt Engineering for Sustainable Inference of LLMs. Specifically, we propose a complexity-aware pipeline where the user queries are parsed, the semantic prompt is compressed, the optimization process is validated, and the inference settings are adapted. We seek to minimize the tokens used, the latency incurred, and the energy consumed, all while keeping a decent response quality.

The major contributions of this work are:

- A complexity-aware query analysis mechanism for adaptive optimization policy selection.
- A semantic prompt optimization pipeline combining prompt decomposition, compression, and validation.
- An adaptive prompt generation strategy that dynamically adjusts inference parameters according to task complexity.
- A monitoring and evaluation framework for measuring token usage, latency, and estimated energy consumption.
- Experimental validation demonstrating substantial reduc-

tions in token consumption and inference latency while preserving output quality across multiple task categories.

II. Research Objectives

The key aim of this study is to devise a framework with an energy-conscious approach that makes LLM inferencing more efficient without compromising the response quality.

Specific objectives include:

1) To assess the complexity of user queries before inferencing and categorize tasks according to their computational demands.
2) To compress prompts semantically, reduce redundancies, and cut down context.
3) To adaptively produce prompts that correspond to task complexity and quality needs.
4) To dynamically tune the parameters required for inference, such as token budget and temperature.
5) To track token usage, response time, and energy-based metrics throughout the inference process.
6) To assess the efficiency-quality tradeoff using a comparative benchmarking method relative to a control group.

III. Literature Review

Due to the fast development of LLMs, there is an ever-growing need to address issues related to computational efficiency and energy sustainability. There have been some attempts at finding ways to reduce the resource consumption of LLMs using various techniques, such as optimization of models and prompts.

Rubei et al. (2025) [1] performed an empirical study aimed at exploring the effect of prompt engineering practices on the energy footprint and computational sustainability of LLMs. The technique used by the researchers comprised testing a local model using various structurally different prompts within an isolated experimental environment. Speaking of the specifics of conducting experiments and synthesizing data, the researchers considered the following basic metrics by randomly selecting 1,000 Java code fragments in the CodeXGLUE benchmark. The number of queries was limited to 75 per prompt for a quantized Llama 38B Instruct model under observation by the CodeCarbon library. In terms of the results, it became evident that PET type makes a significant difference in energy usage profiles, with conventional few-shot baseline models requiring substantially more energy (0.000054kWh) compared to zero-shot baselines (0.000016kWh). However, separating elements of the prompt structure with explicit HTML-like tags in conjunction with contextualization of the latter provides for a significant reduction in power consumption at the inference stage, which decreases inference energy costs by 83% for few-shot and even up to 99% for one-shot models without degrading performance of the underlying code. One of the major advantages of the paper is its holistic approach to the issue in question. Namely, it has shown that there is indeed positive synergy between environmental concerns and computing efficiency, as tag customization enhances performance in terms of increased number of exact matches in zero-shot (23% increase) and decreased edit distance in few-shot models (70% decrease). Nevertheless, it should be noted that the main drawback of the paper is its narrow scope.

In their research, Adamska et al. (2025) [2] employed a scientific, metrics-based approach to investigate the prompt-driven energy consumption of LLM inference. The process of the research included creating an isolated profiling environment based on the ZEUS library to remove any external disturbances and accurately measure the GPU power draw in real time for various interaction parameters. As for the experiment design and data synthesis stage, the authors examined performance phase metrics on three 7-billion-parameter models, including Mistral 7B, Gemma 7B, and Vicuna 7B. These models have been tested using a set of 9,000 unique prompts with question answering, sentiment analysis, and text generation. It was found that the length of the response has the largest influence on the energy consumption and demonstrates a correlation of 0.9, while the prompt length shows insignificant effects (between 0.08 and 0.19). It demonstrates the influence that semantic intent and verbosity have on the physical energy required to execute tasks. One of the major advantages of the research is its detailed behavioral analysis. This provides practical lexical knowledge on issues such as the impact of swapping the more liberal verb ”explain” for the more restrictive term ”classify”, which results in a decrease in physical energy ranging from 50% to 68% without changing the structure of the neural net model being used. The drawback of the paper, in turn, is the fact that it mainly serves as an architectural description of existing systems rather than proposing new solutions.

In an effort to investigate the energy considerations and environmental sustainability of LLM inference optimizations, Fernandez et al. (2025) [3] undertook an empirical study to understand the energy considerations of inference optimizations. In this regard, the approach was to develop a profiling system by leveraging the CodeCarbon library and measuring the power usage of isolated hardware GPUs based on differences in data dimensionality, decoding methods, and deep learning backend software frameworks. Regarding experiment design and data synthesis, the authors have experimented with decoder-only models from 1 billion to 32 billion parameters. It includes the Llama 3.1 and Qwen lineages of large language models. It helps simulate large-scale batched operations in hardware such as the Nvidia RTX A6000 GPU for establishing upper and lower boundaries of operational energy use. Baseline models with no optimizations, including the native implementation of PyTorch, use up to 506.5% more energy than expected on conversations. Properly implemented software-level optimizations, like vLLM, paired with CUDA Graph serialization, considerably reduce memory-bound limitations to cut total inference energy requirements by up to 73%. One advantage of this research survey is the practical way it categorizes tokens' input-output distribution. It provides examples

of highly scalable and specifically designed solutions, such as the application of speculative decoding in the low-batch regime with a continuous batching approach to achieve energy reductions of up to 29.1%. A disadvantage of this paper is that, unlike many others, it serves mainly as a characterizing study with respect to the GPU's energy footprint alone, without providing any novel architecture.

Wilhelm et al. (2025) [4] conducted an analysis study to evaluate the trade-offs between energy consumption and accuracy in test-time computing techniques used during language model inference. Their objective was to push sustainability metrics like energy per token to become popular. The methodological approach comprised the assessment of computation operations' profiles for models of different sizes using the MMLU evaluation framework. Input/output tokens were examined to reveal the non-linear hardware energy cost functions associated with transformers. As for the experimental design and data synthesis, the authors looked into how advanced reasoning techniques such as chain-of-thought prompting and majority voting influenced the efficiency of small vs. large models. However, findings pointed out that although increasing compute at inference time could make smaller models nearly match the performance achieved by more extensive models, such prompting techniques come with considerable extra energy costs. Moreover, the study indicates that the amount of energy needed to generate each token does not stay constant among models with similar parameter sizes. They exhibit notable differences in their hardware efficiency with varying tasks. One notable contribution of the research is the long-term view of implementing a new query-routing paradigm that employs operational curves to dynamically control the depth of token generation and strikes a balance between linguistic quality and the environmental cost. Nevertheless, one drawback of the paper is that it serves as nothing more than a theoretical framework rather than an actual cross-platform scheduling mechanism.

Cappendijk, de Reus, and Oprescu (2025) [5] examined the effect of prompt tuning on the energy efficiency of generated Python code by an LLM. The approach entailed identifying the impacts of green programming instructions on the footprint characteristics depending on problem complexity. Abstract syntax trees were applied to track the differences in the structural form between the baseline and optimized code. Regarding the experimental design and data synthesis, Cappendijk et al. tested 60 configurations on five different LLMs, such as Code Llama-70b, DeepSeek-Coder-33b, and LeetCode challenges, using 50 iterations. These results were further explained using a physical machine's performance statistics with Intel RAPL energy registers and maximum memory consumption recorded with GNU time. The findings revealed a considerable benchmark effect under particular restrictions; specifically, Code Llama-70b-Python reduced the amount of execution energy used in sorting libraries by 59.4%, yet showed a completely static syntax (100% similarity) on finding the median of two sorted arrays. An environmental noise floor is thus estimated to range from -4.9% to $+0.5\%$. Additionally, the research revealed the evident threshold impact with regard to instruction adherence, when the application of specific constraints, such as the for-loop, resulted in inefficient code generation patterns, compelling the DeepSeek-Coder-33b-base to replace concise array operations with three loops, resulting in an astronomical 465.5% increase in energy expenditure. An advantage of this investigation is the multidimensional framework of the assessment scheme encompassing both AST-based and telemetry metrics, which indicates that loop constraints help optimize code not directly but through the implicit application of high-performance internal standard library functions. A drawback of the paper under consideration is its strong dependence on the deterministic decoding architecture based on greedy decoding and static low temperatures combined with hardware-related restrictions, meaning that no tests were conducted outside one consumer laptop.

Model routing and inference have also been looked at within recent literature on sustainability in AI. Small models perform simple tasks, whereas large models perform tasks that are reasoning-intensive. This kind of approach is effective but requires the management of multiple deployed models and routing mechanisms.

Another problem with most current research is the absence of an integrated approach that integrates elements such as complexity analysis, prompt dissection, semantic compression, quality testing, adaptive response generation, and performance monitoring into one single pipeline system. The majority of research conducted to date tends to concentrate on just one aspect of optimization.

The proposed study intends to fill these gaps by designing a novel Energy-Aware Prompt Engineering Framework that incorporates optimization at several levels, checks the preservation of semantics using embedding similarity, and measures the gains in efficiency compared to an inefficient baseline. What is different about the suggested framework from existing frameworks is that it offers a complete framework for sustainable LLM inference.

IV. Methodology

A. Overview

The Energy-Aware Prompt Engineering Framework is expected to save computational costs and energy consumption on LLM inference while maintaining response quality. This framework adopts a framework structure that has eight layers, which include query analysis, prompt decomposition, optimization, validation, adaptive prompt generation, inference run, monitoring, and evaluation.

B. Query Complexity Analyzer

The first step involves analyzing the incoming request and determining the level of complexity. This is achieved through

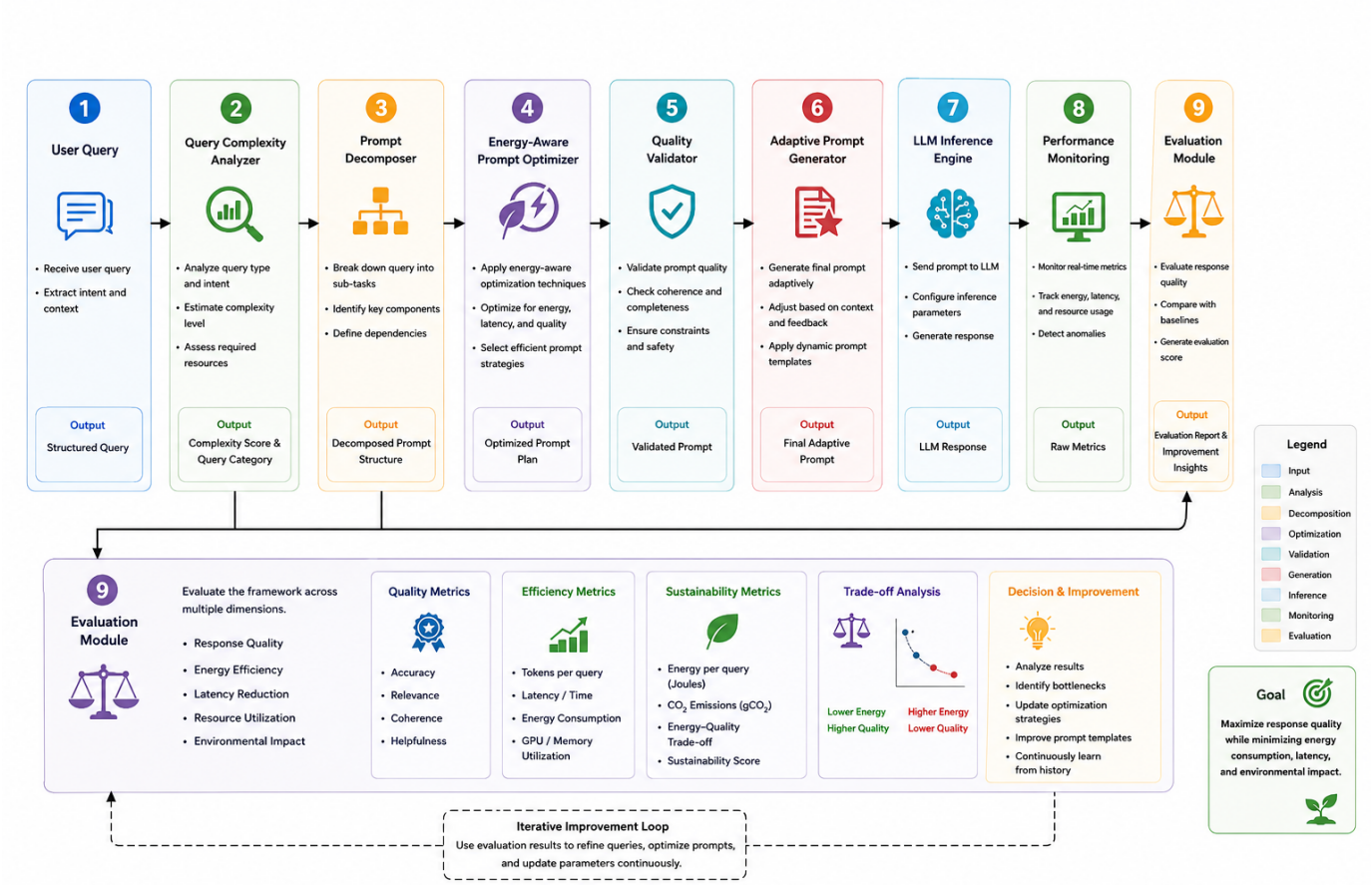

Fig. 1. Energy-Aware Prompt Engineering Framework

several signals, such as word count, sentence count, reasoning, safety, context size, and ambiguity.

The following complexity levels are assigned to the request by the analyzer:

- Low
- Medium
- High
- Critical

These complexity levels are associated with optimization policies dictating the extent to which the prompt can be optimized.

C. Prompt Decomposer

The decomposer converts the initial input into a well-structured format that consists of:

- Intent of the user
- Request
- Entities
- Constraints
- Context

This step guarantees the preservation of all vital information before optimization.

D. Energy-Aware Prompt Optimizer

The optimization process is carried out using a combination of semantic optimization and rule-based optimizations.

The optimizer applies:

- Semantic prompt compression
- Removal of filler phrases
- Removal of redundancies
- Reduction of context length
- Whitelines normalization

Depending on the chosen policy, optimization levels vary.

E. Quality Validator

There can be a drift in meaning because of prompt compression. Thus, to ensure that the compressed prompt has not lost its original meaning, a validation step is required.

For validation, the following is used:

- Lexical constraints
- Embedding similarity evaluation
- Similarity threshold evaluation

If the similarity value is less than the threshold value, then the original prompt will remain unchanged.

F. Adaptive Prompt Generator

Inference configuration will be generated in real time, dynamically based on task complexity.

Parameters include:

- Token budget cap
- Temperature
- Top-p sampling
- Response style templates

For instance, lower complexity definition tasks get smaller token budget caps, whereas coding and reasoning tasks get higher generation limits.

G. LLM Inference Engine

The inference step uses language models hosted by Ollama.

Models involved in the process are:

- Qwen2.5 0.5B (inference and optimization)
- Nomic-Embed-Text (validation)
- Qwen3.5 4B (final inference)

With such a division of work, small models can

H. Performance Monitoring

The Monitoring component measures the following parameters:

- Number of prompt tokens
- Number of completion tokens
- Total number of tokens
- Latency
- Proxy for energy consumption
- Joules estimated

These metrics provide the basis for evaluating efficiency improvements.

I. Evaluation Strategy

Comparison of the enhanced pipeline with the baseline approach involves measuring its performance relative to the baseline system, whereby the prompt is delivered straight to the LLM without optimization.

The following evaluation criteria will be considered:

- Token reduction (%)
- Latency reduction (%)
- Energy savings (%)
- Quality maintenance

V. Experimental Setup

A. Dataset and Benchmark Tasks

The developed methodology was analyzed on the basis of a benchmark dataset that explicitly aimed at representing typical LLM use cases. Prompts used in the benchmarks were saved under `data/benchmark_prompts.json` and covered a wide range of tasks, from definition generation, education-based summarization, mathematical reasoning, programming help, safety-critical questions, to context-dependent information retrieval.

As part of the analysis carried out within this work, four benchmark categories were chosen:

- Definition Tasks
- Education-Based Summarization
- Mathematical Reasoning
- Programming Help

The set of benchmarks included 23 cases used for verifying the proper functionality of the complexity classifier, the prompt optimizer, and the adaptive prompt generation method. Each benchmark case provided expected complexities, policies for optimization, and the behavior of prompt generation.

B. Data Preprocessing

The inputs for all queries before performing inference had undergone preprocessing using the novel Energy-aware Prompt Engineering Framework, which involved:

- Analysis of query complexity
- Prompt decomposition
- Semantically compacted prompts
- Removing redundancy
- Context reduction
- White space standardization
- Embedding similarity-based quality check
- Generation of adaptive prompts

Preprocessing was employed in an effort to minimize wastage of tokens while maintaining the semantics of the user query intact.

C. Software Tools and Libraries

The framework was developed in Python 3.10+ and tested with a modular pipeline approach. The main components used were as follows:

- Python 3.10+
- Ollama to deploy an LLM locally
- PyTest to validate and test benchmarks
- Custom-built modules to conduct complexity analysis, prompt optimization, adaptive prompt generation, monitoring, and evaluation

Inferences were made with locally deployed models by Ollama, which meant no dependence on external API services.

D. Language Models

Three language models were used in the system:

- **Qwen2.5 0.5B**: Query analysis and prompt optimization
- **Nomic-Embed-Text**: Validation of semantic similarity
- **Qwen3.5 4B**: Response generation

Using different models in this way makes it possible to have

E. Hardware Environment

The experiments were performed locally using Ollama and performing CPU-based inference. As we did not have access to GPU energy data directly, we calculated it based on energy proxy values derived from tokens and execution time.

F. Evaluation Metrics

The efficiency of the above framework was tested using a baseline approach where the users' requests were sent directly to the language model without any optimization.

These measures were used during the experiment:

- Prompt Tokens
- Completion Tokens
- Total Tokens
- Inference Latency (ms)
- Energy Proxy
- Energy Estimate

To calculate the extent to which the efficiency has been improved, we computed the following measures:

- Token Reduction (%)
- Latency Reduction (%)
- Energy Reduction (%)
- Quality Preservation

Quality Preservation was verified through a comparison between optimized results and baseline results using an embedding similarity check.

VI. Results

A. Benchmark Evaluation

The framework was evaluated using four benchmark categories representing common LLM workloads:

- Definition Tasks
- Educational Summarization
- Mathematical Reasoning
- Coding Assistance

TABLE I
BENCHMARK RESULTS

Benchmark	Baseline Tokens	Optimized Tokens	Token Reduction
TCP Definition	187	161	13.9%
OS Study Notes	982	317	67.7%
Project Assignment	1034	332	67.9%
Reasoning			
Dijkstra Coding	2586	636	75.4%

TABLE II
LATENCY COMPARISON

Benchmark	Baseline (ms)	Optimized (ms)	Reduction
TCP Definition	9020	3908	56.7%
OS Study Notes	14880	5907	60.3%
Reasoning Task	15206	5921	61.1%
Coding Task	35624	10265	71.2%

Figure 2 illustrates the empirical assessment of performance between Baseline and Optimized configurations on a generic networking query prompt ("What is TCP three-way handshake? Explain in 3-4 sentences with a simple example.") via an evaluation that examines how changes in the system can affect the effectiveness of language models through the use of three key diagnostics channels: Latency (ms), Tokens, and Energy. In the analysis, an acute optimization effect is observed under the selected parameterization; for example, the configuration saw a significant reduction in Latency, dropping from a baseline level of 9,020 ms to an optimized latency of 3,908 ms. The runtime increase is coupled with a trend towards a slight reduction in generated token size, achieving 161 tokens against the initial baseline level of 187 tokens. Meanwhile, resource usage by the physical systems is illustrated to decline, with the Energy proxy measure going down from a baseline value of 0.1496 to an optimized figure of 0.1288.

Figure 3, on the other hand, illustrates an even more empirical examination conducted using an extremely difficult, multi-faceted Operating Systems command template ("Generate concise study notes about the concepts of Process, Thread, Context Switching, and Deadlock"). The figure shows structural differences in a situation where conditions are identical, with metric performance assessed for a very large generation output. From the results, it is evident that the most aggressive optimizations are seen throughout all experimentations; Latency is significantly reduced from a very high benchmark value of 14,880 ms to an optimized value of 5,907 ms. The reduction is caused by the severe compression in output size, since the number of tokens reduces significantly from 982 tokens to 317 tokens. Given that the generation size is drastically reduced, the system resources cost decreases dramatically in a linear trend, from a high benchmark of 0.7856 to an optimized value of 0.2536.

The System Metrics in Figure 4 utilize an advanced, extremely conversational Software Testing prompt template that places the performance of the system within context. Such metrics are characterized by a distinctly asymmetric optimization in terms of their response to this particular set of lengthy conversational parameters. Although the systematic modification of workflows helps ensure the optimization of the number of tokens used during generation, the total size of the output is successfully reduced from 626 to 370. The resulting optimization in terms of tokens used does not result in any significant hardware speedup since Latency only shows a slight drop from 9,377.2 ms to 8,769.2 ms. Despite such a lackluster performance in terms of hardware optimizations, the physical Energy metrics show an effective decrease, dropping from a starting point of 0.5008 to 0.2960.

TABLE III
AGGREGATE PERFORMANCE

Metric	Baseline Average	Optimized Average	Improvement
Latency	18.68 s	6.50 s	65.2%
Tokens	1197	362	69.8%
Energy Proxy	0.9578	0.2892	69.8%

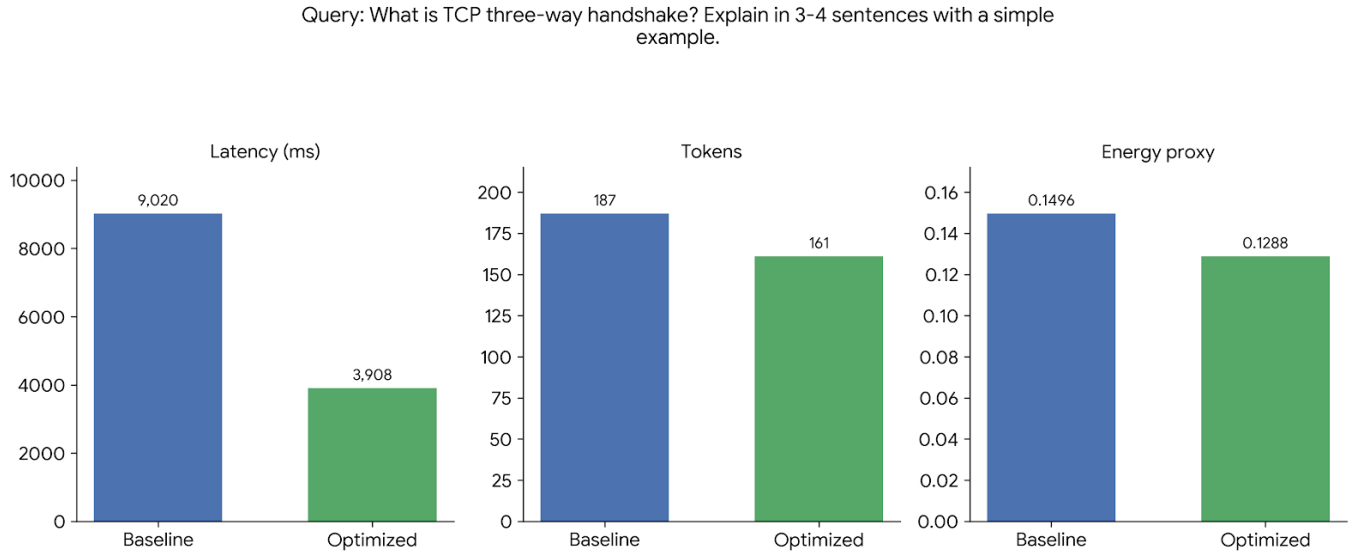

Fig. 2. Simple Query Results

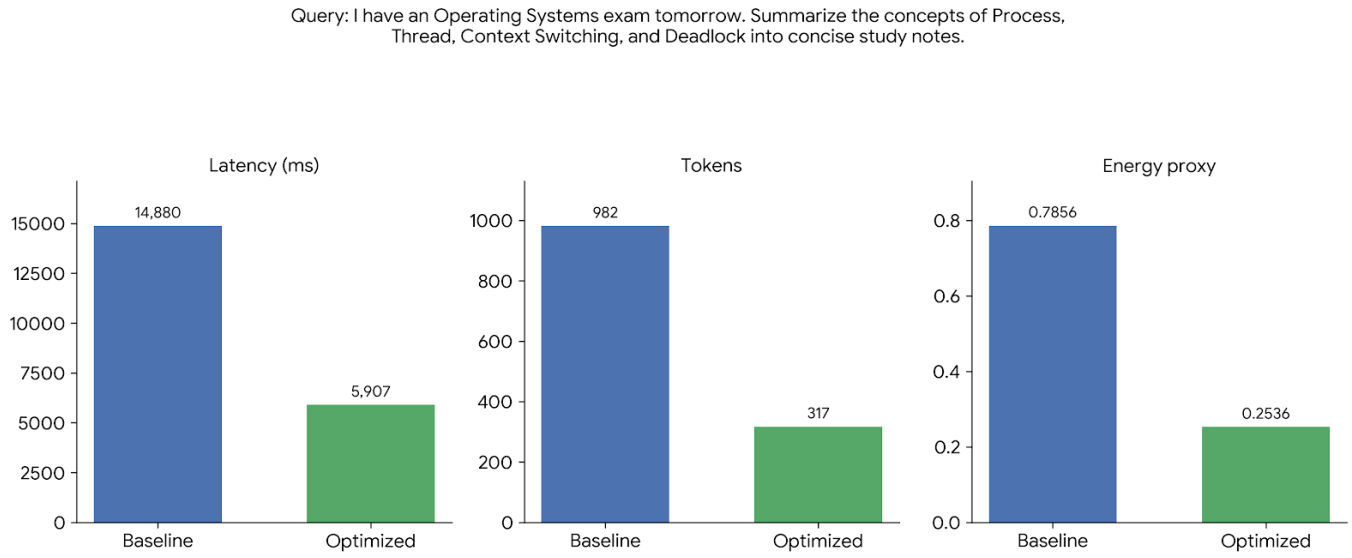

Fig. 3. Medium Query Results

B. Quality Assessment

Quality was maintained in three out of four categories after optimization.

Success stories:

- Definitions
- Educational Summarization
- Programming

The only problem category was the one that required combinatorics because of the wrong task definition and, therefore, excessive token compression.

In total, the framework succeeded at maintaining quality in 75% of benchmarks.

VII. Discussion

The findings of the experiment show that optimization at the prompt level can considerably enhance the efficiency of LLMs. On average, the proposed approach decreased the number of tokens by 69.8% and latency by 65.2%, which means that a significant part of the computational cost was due to the ineffective use of prompts and generation budgets.

The most significant gains were registered for coding and summarization tasks. This was possible thanks to the efficient allocation of token budgets, which helped restrict unnecessary token generation without sacrificing the core information

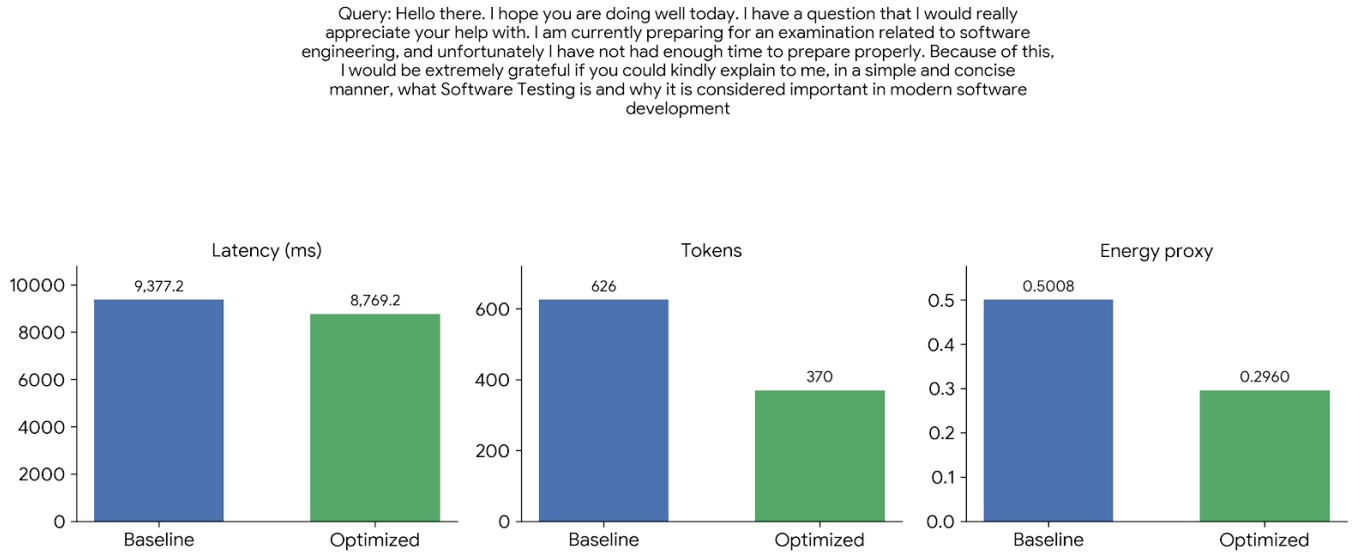

Fig. 4. Complex Query Results

demanded by the user. It is worth noting that the coding benchmark demonstrated the highest accuracy of optimized generation.

In turn, the reasoning challenge showed one of the key drawbacks of our approach – a complexity analyzer misclassified a mathematical reasoning question into a low complexity educational request category, which resulted in the use of an aggressively applied compression method, leading to an incomplete answer. Therefore, energy savings may be considered insignificant if the response quality is significantly reduced.

It is worth noting that our approach allows for creating a more holistic approach compared to other prompt compression methods that are known from prior literature in the field. In particular, our solution integrates complexity analysis, semantic optimization, validation, adaptive response generation, and system monitoring. Importantly, embedding-based validation plays an essential role in mitigating the risk of prompt semantic shift during the compression process.

In conclusion, it is possible to say that our research shows that prompt engineering may become a valuable tool for implementing sustainable AI deployment solutions without changes to the model architecture or costly training sessions.

VIII. Conclusion

In this research, an Energy-Aware Prompt Engineering Framework for Sustainable Large Language Model Inference is proposed. The Energy-Aware Prompt Engineering Framework combines several key components such as complexity-aware query analysis, semantic prompt engineering, quality check, adaptive prompt engineering, performance analysis, and comparative evaluation to form an efficient pipeline.

Through experimental evaluations, we observed significant reductions in terms of token utilization, latency, and energy consumption. On average, the Energy-Aware Prompt Engineering Framework reduced tokens by 69.8% and latency by 65.2%.

It is apparent from the results that considerable efficiency improvements can be realized solely by using prompt engineering techniques intelligently, independent of any changes to the existing architecture of language models. This solution serves as one possible step towards the greater objective of developing Green AI solutions.

Even though there are some obstacles in the way of solving reasoning problems, this framework lays a good foundation for future research endeavors.

IX. Limitations

Even with the promising results generated through the implementation of the Energy-aware Prompt Engineering Framework, there are still some limitations that need to be noted. One major limitation is the measurement of the energy consumption used in this research. In this case, all the energy measurement provided here is based on token-based energy estimation proxies and CPU-based energy estimation. This means that the energy consumption data used here is not entirely accurate as far as real-world energy consumption is concerned.

The other limitation is about the scale of the evaluation. In this experiment, only a small set of benchmarked tasks, including those related to definition generation, summarization, reasoning, and coding, was analyzed. Although this provided insight into the performance of the proposed framework, a larger set

of benchmarks will be required to generalize its performance across various applications.

The existing quality assessment approach is subject to several limitations as well. The quality of responses was mostly measured via output comparisons and human inspection without comprehensive human assessment or more sophisticated automated approaches. Thus, there might be some nuances regarding response usefulness, accuracy, or users' satisfaction that cannot be detected during the process of evaluation.

Another aspect that can be considered a limitation is related to the use of a single model for generating responses. Adaptive prompts optimize this process; however, the performance of such a framework can be improved by adding some model selection and routing approaches. Lastly, a limitation identified in the proposed framework pertains to its poor performance on mathematically oriented benchmarks.

X. Future Work

Several areas are ripe for exploration that may result in enhancements and advancements to this framework. A significant area would be the creation of a more advanced reasoning detector that can recognize when a query requires mathematical, logical, or multiple-step analysis prior to optimization. This enhancement will permit the appropriate allocation of computing resources and prevent over-compression of requests involving reasoning. Future iterations of the framework may include multi-model routing, whereby simpler queries are processed using light models and complex queries are directed towards more powerful models.

The use of retrieval augmented generation technology and intelligent context selection mechanisms may further cut down on unnecessary calculations.

One additional promising area includes the replacement of proxy methods for estimating energy consumption with direct measurements from GPUs and other types of inference hardware. Such an approach would allow for better understanding and measuring the impact on the environment caused by optimizing prompting techniques. Human judgments and machine evaluation via LLMs can also be used in future research to provide a comprehensive overview of response quality and user experience.

Lastly, more extensive experiments on diverse sets of benchmarks and actual workloads, as well as various language models, could further help evaluate the proposed framework. Moreover, introducing reinforcement learning and adapting policies based on the current state could potentially allow for discovering the most efficient prompts.

References

[1] Riccardo Rubei, Aicha Moussaid, Claudio Di Sipio, and Davide Di Ruscio. Prompt engineering and its implications on the energy consumption of large language models. In *2025 IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS)*, page 60–67. IEEE, April 2025.

[2] Marta Adamska, Daria Smirnova, Hamid Nasiri, Zhengxin Yu, and Peter Garraghan. Green prompting: Characterizing prompt-driven energy costs of llm inference, 2026.

[3] Jared Fernandez, Clara Na, Vashisth Tiwari, Yonatan Bisk, Sasha Luccioni, and Emma Strubell. Energy considerations of large language model inference and efficiency optimizations. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32556–32569, Vienna, Austria, July 2025. Association for Computational Linguistics.

[4] Patrick Wilhelm, Thorsten Wittkopp, and Odej Kao. Beyond test-time compute strategies: Advocating energy-per-token in llm inference, 2026.

[5] Tom Cappendijk, Pepijn de Reus, and Ana Oprescu. An exploration of prompting llms to generate energy-efficient code. In *2025 IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS)*, pages 31–38, 2025.